

— 2교시 정리

데이터 조회와 전처리

이 시간에 쓴 코드와 실행 결과

2.1

열 고르기

형식

```
df[['열1', '열2']]          → 그 열만 남긴다  
df.drop(['열'], axis=1)     → 그 열을 버린다
```

예제

```
cols = ['Order ID', 'Region', 'Sales']  
print(df[cols].shape)  
print(df.drop(['Row ID'], axis=1).shape)
```

실행 결과

```
(10239, 3)  
(10239, 20)
```

`axis=1` 은 열 방향을 뜻한다. `axis=0` 은 행 방향이다.

2.2

조건으로 거르기

형식

<code>df[조건]</code>	→ 조건이 참인 행만 남는다
<code>df[(조건1) & (조건2)]</code>	→ 그리고
<code>df[(조건1) (조건2)]</code>	→ 또는

예제

```
print(len(df[df['Sales'] > 1000]))
```

```
print(len(df[(df['Sales'] > 1000) &  
              (df['Profit'] < 0)]))
```

실행 결과

476

80

조건이 두 개 이상이면 **각 조건을 괄호로 감싼다**. 빠뜨리면 오류가 난다.

2.3

여러 값 · 범위 · 글자

함수	기능
<code>.isin([...])</code>	목록 중 하나와 같은가
<code>.between(a, b)</code>	a 이상 b 이하인가
<code>.str.contains('...')</code>	글자가 들어 있는가

예제

```
print(len(df[df['Region'].isin(['West', 'East'])]))
print(len(df[df['Sales'].between(100, 500)]))
print(len(df[df['Product Name']
                .str.contains('Chair', na=False)]))
```

실행 결과

6270
2649
697

`str.contains()` 에 `na=False` 를 넣지 않으면 빈 칸에서 오류가 난다.

2.4

정렬

형식

`df.sort_values('열', ascending=False)` → 큰 값부터

예제

```
df.sort_values('Sales', ascending=False)[  
    ['Sub-Category', 'Sales']].head(3)
```

실행 결과

	Sub-Category	Sales
7547	Tables	104835.00
227	Machines	22638.48
5920	Copiers	17499.95

`ascending=True` (기본값) 는 작은 값부터 정렬한다.

2.5

결측치 처리

방법	코드	언제
세기	<code>.isna().sum()</code>	항상 먼저
버리기	<code>.dropna(subset=['열'])</code>	무작위 · 양이 적을 때
채우기	<code>.fillna(값)</code>	근거가 있을 때

예제

```
print(df['Ship Date'].isna().sum())
print(len(df))
print(len(df.dropna(subset=['Ship Date'])))
```

실행 결과

321
10239
9918

`dropna()` 한 줄로 **321행이 사라졌다**. 이 결측치는 반품 건에 몰려 있었으므로, 지우면 반품률이 실제보다 낮게 나온다.

2.6

중복 처리

형식

<code>df.duplicated().sum()</code>	→ 모든 칸이 같은 행이 몇 개인가
<code>df.drop_duplicates()</code>	→ 그 행을 지운다

예제

```
print(df.duplicated().sum())  
print(len(df.drop_duplicates()))
```

실행 결과

```
45  
10194
```

`subset=['Order ID']` 처럼 특정 열만 기준으로 지우면 **같은 주문의 다른 품목이 통째로 사라진다**. 주문번호 반복은 중복이 아니다.

2.7

자료형 바꾸기

형식

<code>df['열'].astype(int)</code>	→ 정수로
<code>pd.to_numeric(df['열'], errors='coerce')</code>	→ 숫자로, 안 되는 값은 결측치로
<code>pd.to_datetime(df['열'])</code>	→ 날짜로

예제

```
s = pd.Series(['10', '20', '삼십'])  
print(pd.to_numeric(s, errors='coerce'))
```

실행 결과

```
0    10.0  
1    20.0  
2     NaN  
dtype: float64
```

`errors='coerce'` 는 바꿀 수 없는 값을 결측치로 만든다. 이것을 빼면 값 하나 때문에 전체가 오류로 멈춘다.

2.8

날짜에서 조각 꺼내기

날짜로 읽힌 열은 `.dt` 를 통해 연 · 월 · 요일을 꺼낼 수 있다.

예제

```
print(df['Order Date'].dt.year.head(2).tolist())
print(df['Order Date'].dt.month.head(2).tolist())
print(df['Order Date'].dt.to_period('M')
      .head(2).astype(str).tolist())
```

실행 결과

```
[2023, 2023]
[1, 1]
['2023-01', '2023-01']
```

글자로 읽힌 날짜에는 `.dt` 를 쓸 수 없다. 1교시의 `parse_dates` 가 필요한 이유다.

한 장 정리

`df[['열1', '열2']]` · `.drop(['열'], axis=1)`

열 고르기 · 버리기

`df[df['열'] > 값]`

조건으로 거르기

`(조건1) & (조건2)` · `|`

조건 잇기 — 각 조건에 괄호

`.isin()` · `.between()` · `.str.contains()`

여러 값 · 범위 · 글자

`.sort_values('열', ascending=False)`

정렬

`.isna().sum()` · `.dropna()` · `.fillna()`

결측치 세기 · 버리기 · 채우기

`.duplicated().sum()` · `.drop_duplicates()`

중복 세기 · 지우기

`.astype()` · `pd.to_numeric(errors='coerce')`

자료형 바꾸기

`.dt.year` · `.dt.month` · `.dt.to_period('M')`

날짜 조각